Machine-Level Programming IV: Data

CSE 238/2038/2138: Systems Programming

Instructors:

Zuhal Altuntaş & Birol Gençyılmaz

Slides adapted from Bryant & O'Hallaron's slides

Array Example

Practice Problem 3.36 (solution page 377)

Consider the following declarations:

```
int    P[5];
short  Q[2];
int    **R[9];
double *S[10];
short  *T[2];
```

Fill in the following table describing the element size, the total size, and the address of element i for each of these arrays.

Array	Element size	Total size	Start address	Element i
P	_____	_____	x_P	_____
Q	_____	_____	x_Q	_____
R	_____	_____	x_R	_____
S	_____	_____	x_S	_____
T	_____	_____	x_T	_____

Array Example

Practice Problem 3.36 (solution page 377)

Consider the following declarations:

```
int    P[5];
short  Q[2];
int    **R[9];
double *S[10];
short  *T[2];
```

Fill in the following table describing the element size, the total size, and the address of element *i* for each of these arrays.

Array	Element size	Total size	Start address	Element *i*
P	4	20	x_P	$x_p + 4i$
Q			x_Q	
R			x_R	
S			x_S	
T			x_T	

Array Example

Practice Problem 3.36 (solution page 377)

Consider the following declarations:

```
int    P[5];
short  Q[2];
int    **R[9];
double *S[10];
short  *T[2];
```

Fill in the following table describing the element size, the total size, and the address of element *i* for each of these arrays.

Array	Element size	Total size	Start address	Element *i*
P	4	20	x_P	$x_p + 4i$
Q	2	4	x_Q	$x_Q + 2i$
R			x_R	
S			x_S	
T			x_T	

Array Example

Practice Problem 3.36 (solution page 377)

Consider the following declarations:

```
int    P[5];
short  Q[2];
int    **R[9];
double *S[10];
short  *T[2];
```

Fill in the following table describing the element size, the total size, and the address of element i for each of these arrays.

Array	Element size	Total size	Start address	Element i
P	4	20	x_P	$x_p + 4i$
Q	2	4	x_Q	$x_Q + 2i$
R	8	72	x_R	$x_R + 8i$
S			x_S	
T			x_T	

Array Example

Practice Problem 3.36 (solution page 377)

Consider the following declarations:

```
int    P[5];
short  Q[2];
int    **R[9];
double *S[10];
short  *T[2];
```

Fill in the following table describing the element size, the total size, and the address of element *i* for each of these arrays.

Array	Element size	Total size	Start address	Element *i*
P	4	20	x_P	$x_p + 4i$
Q	2	4	x_Q	$x_Q + 2i$
R	8	72	x_R	$x_R + 8i$
S	8	80	x_S	$x_S + 8i$
T			x_T	

Array Example

Practice Problem 3.36 (solution page 377)

Consider the following declarations:

```
int    P[5];
short  Q[2];
int    **R[9];
double *S[10];
short  *T[2];
```

Fill in the following table describing the element size, the total size, and the address of element *i* for each of these arrays.

Array	Element size	Total size	Start address	Element *i*
P	4	20	x_P	$x_p + 4i$
Q	2	4	x_Q	$x_Q + 2i$
R	8	72	x_R	$x_R + 8i$
S	8	80	x_S	$x_S + 8i$
T	8	16	x_T	$x_T + 8i$

Array Example

Practice Problem 3.37 (solution page 377)

Suppose x_P, the address of short integer array `P`, and long integer index i are stored in registers `%rdx` and `%rcx`, respectively. For each of the following expressions, give its type, a formula for its value, and an assembly-code implementation. The result should be stored in register `%rax` if it is a pointer and register element `%ax` if it has data type `short`.

Expression	Type	Value	Assembly code
`P[1]`			
`P + 3 + i`			
`P[i * 6 - 5]`			
`P[2]`			
`&P[i + 2]`			

Array Example

Practice Problem 3.37 (solution page 377)

Suppose x_p, the address of short integer array P, and long integer index i are stored in registers %rdx and %rcx, respectively. For each of the following expressions, give its type, a formula for its value, and an assembly-code implementation. The result should be stored in register %rax if it is a pointer and register element %ax if it has data type short.

X_p → (%rdx)
i → %rcx
Dest → %rax

Expression	Type	Value	Assembly code
P[1]	short	M[x_p + 2]	movw 2(%rdx), %ax
P + 3 + i			
P[i * 6 - 5]			
P[2]			
&P[i + 2]			

Array Example

Practice Problem 3.37 (solution page 377)

Suppose x_p, the address of short integer array P, and long integer index i are stored in registers %rdx and %rcx, respectively. For each of the following expressions, give its type, a formula for its value, and an assembly-code implementation. The result should be stored in register %rax if it is a pointer and register element %ax if it has data type short.

X_p → (%rdx)
i → %rcx
Dest → %rax

Expression	Type	Value	Assembly code
P[1]	short	M[x_p + 2]	movw 2(%rdx), %ax
P + 3 + i	short *	x_p + 6 + 2i	leaq 6 (%rdx,%rcx,12), %rax
P[i * 6 - 5]			
P[2]			
&P[i + 2]			

Array Example

Practice Problem 3.37 (solution page 377)

Suppose x_p, the address of short integer array `P`, and long integer index i are stored in registers `%rdx` and `%rcx`, respectively. For each of the following expressions, give its type, a formula for its value, and an assembly-code implementation. The result should be stored in register `%rax` if it is a pointer and register element `%ax` if it has data type `short`.

X_p → (%rdx)
i → %rcx
Dest → %rax

Expression	Type	Value	Assembly code
`P[1]`	short	M[x_p + 2]	movw 2(%rdx), %ax
`P + 3 + i`	short *	x_p + 6 + 2i	leaq 6 (%rdx,%rcx,2), %rax
`P[i * 6 - 5]`	short	M[x_p + 12i - 10]	movw -10(%rdx,%rcx,12), %ax
`P[2]`			
`&P[i + 2]`			

Array Example

Practice Problem 3.37 (solution page 377)

Suppose x_p, the address of short integer array P, and long integer index i are stored in registers %rdx and %rcx, respectively. For each of the following expressions, give its type, a formula for its value, and an assembly-code implementation. The result should be stored in register %rax if it is a pointer and register element %ax if it has data type short.

X_p → (%rdx)
i → %rcx
Dest → %rax

Expression	Type	Value	Assembly code
P[1]	short	M[x_p + 2]	movw 2(%rdx), %ax
P + 3 + i	short *	x_p + 6 + 2i	leaq 6 (%rdx,%rcx,2), %rax
P[i * 6 - 5]	short	M[x_p + 12i - 10]	movw -10(%rdx,%rcx,12), %ax
P[2]	short	M[x_p + 4]	movw 4(%rdx), %ax
&P[i + 2]			

Array Example

Practice Problem 3.37 (solution page 377)

Suppose x_p, the address of short integer array P, and long integer index i are stored in registers %rdx and %rcx, respectively. For each of the following expressions, give its type, a formula for its value, and an assembly-code implementation. The result should be stored in register %rax if it is a pointer and register element %ax if it has data type short.

X_p → (%rdx)
i → %rcx
Dest → %rax

Expression	Type	Value	Assembly code
P[1]	short	M[x_p + 2]	movw 2(%rdx), %ax
P + 3 + i	short *	x_p + 6 + 2i	leaq 6 (%rdx,%rcx,2), %rax
P[i * 6 - 5]	short	M[x_p + 12i - 10]	movw -10(%rdx,%rcx,12), %ax
P[2]	short	M[x_p + 4]	movw 4(%rdx), %ax
&P[i + 2]	short *	x_p + 2i + 4	leaq 4 (%rdx,%rcx,2), %rax

Array Example

Practice Problem 3.38 (solution page 377)

Consider the following source code, where M and N are constants declared with `#define`:

```
long P[M][N];
long Q[N][M];

long sum_element(long i, long j) {
    return P[i][j] + Q[j][i];
}
```

```
long sum_element(long i, long j)
i in %rdi, j in %rsi
1   sum_element:
2     leaq    0(,%rdi,8), %rdx
3     subq    %rdi, %rdx
4     addq    %rsi, %rdx
5     leaq    (%rsi,%rsi,4), %rax
6     addq    %rax, %rdi
7     movq    Q(,%rdi,8), %rax
8     addq    P(,%rdx,8), %rax
9     ret
```

Use your reverse engineering skills to determine the values of M and N based on this assembly code.

Array Example

Practice Problem 3.38 (solution page 377)

Consider the following source code, where M and N are constants declared with `#define`:

```
long P[M][N];
long Q[N][M];

long sum_element(long i, long j) {
    return P[i][j] + Q[j][i];
}
```

```
long sum_element(long i, long j)
i in %rdi, j in %rsi
1   sum_element:
2     leaq    0(,%rdi,8), %rdx
3     subq    %rdi, %rdx
4     addq    %rsi, %rdx
5     leaq    (%rsi,%rsi,4), %rax
6     addq    %rax, %rdi
7     movq    Q(,%rdi,8), %rax
8     addq    P(,%rdx,8), %rax
9     ret
```

Compute 8i →rdx

Use your reverse engineering skills to determine the values of M and N based on this assembly code.

Array Example

Practice Problem 3.38 (solution page 377)

Consider the following source code, where M and N are constants declared with `#define`:

```
long P[M][N];
long Q[N][M];

long sum_element(long i, long j) {
    return P[i][j] + Q[j][i];
}
```

```
long sum_element(long i, long j)
i in %rdi, j in %rsi
1   sum_element:
2     leaq    0(,%rdi,8), %rdx
3     subq    %rdi, %rdx
4     addq    %rsi, %rdx
5     leaq    (%rsi,%rsi,4), %rax
6     addq    %rax, %rdi
7     movq    Q(,%rdi,8), %rax
8     addq    P(,%rdx,8), %rax
9     ret
```

Compute 8i →rdx
Compute 7i →rdx

Use your reverse engineering skills to determine the values of M and N based on this assembly code.

Array Example

Practice Problem 3.38 (solution page 377)

Consider the following source code, where M and N are constants declared with `#define`:

```
long P[M][N];
long Q[N][M];

long sum_element(long i, long j) {
    return P[i][j] + Q[j][i];
}
```

```
long sum_element(long i, long j)
i in %rdi, j in %rsi
1   sum_element:
2     leaq    0(,%rdi,8), %rdx
3     subq    %rdi, %rdx
4     addq    %rsi, %rdx
5     leaq    (%rsi,%rsi,4), %rax
6     addq    %rax, %rdi
7     movq    Q(,%rdi,8), %rax
8     addq    P(,%rdx,8), %rax
9     ret
```

Compute 8i →rdx
Compute 7i →rdx
Compute 7i + j →rdx

Use your reverse engineering skills to determine the values of M and N based on this assembly code.

Array Example

Practice Problem 3.38 (solution page 377)

Consider the following source code, where M and N are constants declared with `#define`:

```
long P[M][N];
long Q[N][M];

long sum_element(long i, long j) {
    return P[i][j] + Q[j][i];
}
```

```
long sum_element(long i, long j)
i in %rdi, j in %rsi
1   sum_element:
2     leaq    0(,%rdi,8), %rdx
3     subq    %rdi, %rdx
4     addq    %rsi, %rdx
5     leaq    (%rsi,%rsi,4), %rax
6     addq    %rax, %rdi
7     movq    Q(,%rdi,8), %rax
8     addq    P(,%rdx,8), %rax
9     ret
```

Compute 8i →rdx
Compute 7i →rdx
Compute 7i + j →rdx
Compute 5j →rax

Use your reverse engineering skills to determine the values of M and N based on this assembly code.

Array Example

Practice Problem 3.38 (solution page 377)

Consider the following source code, where M and N are constants declared with `#define`:

```
long P[M][N];
long Q[N][M];

long sum_element(long i, long j) {
    return P[i][j] + Q[j][i];
}
```

```
long sum_element(long i, long j)
i in %rdi, j in %rsi
1   sum_element:
2     leaq    0(,%rdi,8), %rdx
3     subq    %rdi, %rdx
4     addq    %rsi, %rdx
5     leaq    (%rsi,%rsi,4), %rax
6     addq    %rax, %rdi
7     movq    Q(,%rdi,8), %rax
8     addq    P(,%rdx,8), %rax
9     ret
```

Compute 8i →rdx
Compute 7i →rdx
Compute 7i + j →rdx
Compute 5j →rax
Compute i + 5j →rdi

Use your reverse engineering skills to determine the values of M and N based on this assembly code.

Array Example

Practice Problem 3.38 (solution page 377)

Consider the following source code, where M and N are constants declared with `#define`:

```
long P[M][N];
long Q[N][M];

long sum_element(long i, long j) {
    return P[i][j] + Q[j][i];
}
```

```
long sum_element(long i, long j)
i in %rdi, j in %rsi
1   sum_element:
2     leaq    0(,%rdi,8), %rdx
3     subq    %rdi, %rdx
4     addq    %rsi, %rdx
5     leaq    (%rsi,%rsi,4), %rax
6     addq    %rax, %rdi
7     movq    Q(,%rdi,8), %rax
8     addq    P(,%rdx,8), %rax
9     ret
```

Use your reverse engineering skills to determine the values of M and N based on this assembly code.

Compute 8i →rdx
Compute 7i →rdx
Compute 7i + j →rdx
Compute 5j →rax
Compute i + 5j →rdi
Retrieve $M[x_Q+8*(5j + i)]$ →rax

Array Example

Practice Problem 3.38 (solution page 377)

Consider the following source code, where M and N are constants declared with `#define`:

```
long P[M][N];
long Q[N][M];

long sum_element(long i, long j) {
    return P[i][j] + Q[j][i];
}
```

```
long sum_element(long i, long j)
i in %rdi, j in %rsi
1   sum_element:
2     leaq    0(,%rdi,8), %rdx
3     subq    %rdi, %rdx
4     addq    %rsi, %rdx
5     leaq    (%rsi,%rsi,4), %rax
6     addq    %rax, %rdi
7     movq    Q(,%rdi,8), %rax
8     addq    P(,%rdx,8), %rax
9     ret
```

Use your reverse engineering skills to determine the values of M and N based on this assembly code.

Compute 8i →rdx
Compute 7i →rdx
Compute 7i + j →rdx
Compute 5j →rax
Compute i + 5j →rdi
Retrieve $M[x_Q+8*(5j + i)]$ →rax
Add $M[x_p + 8*(7i+j)]$ →rax

Array Example

Practice Problem 3.38 (solution page 377)

Consider the following source code, where M and N are constants declared with `#define`:

```
long P[M][N];
long Q[N][M];

long sum_element(long i, long j) {
    return P[i][j] + Q[j][i];
}
```

```
long sum_element(long i, long j)
i in %rdi, j in %rsi
1   sum_element:
2     leaq    0(,%rdi,8), %rdx
3     subq    %rdi, %rdx
4     addq    %rsi, %rdx
5     leaq    (%rsi,%rsi,4), %rax
6     addq    %rax, %rdi
7     movq    Q(,%rdi,8), %rax
8     addq    P(,%rdx,8), %rax
9     ret
```

Use your reverse engineering skills to determine the values of M and N based on this assembly code.

8 is scale value
Offset of matrix P 7i + j →
P has 7 columns

Offset of matrix Q 5j + i →
Q has 5 columns

So, M = 5 and N = 7

Compute 8i →rdx
Compute 7i →rdx
Compute 7i + j →rdx
Compute 5j →rax
Compute i + 5j →rdi
Retrieve $M[x_Q+8*(5j + i)]$ →rax
Add $M[x_p + 8*(7i+j)]$ →rax

Structure Example

Practice Problem 3.41 (solution page 379)

Consider the following structure declaration:

```
struct test {
    short *p;
    struct {
        short x;
        short y;
    } s;
    struct test *next;
};
```

This declaration illustrates that one structure can be embedded within another, just as arrays can be embedded within structures and arrays can be embedded within arrays.

The following procedure (with some expressions omitted) operates on this structure:

```
void st_init(struct test *st) {
    st->s.y  = _________;
    st->p    = _________;
    st->next = _________;
}
```

A. What are the offsets (in bytes) of the following fields?

```
p:    _____
s.x:  _____
s.y:  _____
next: _____
```

B. How many total bytes does the structure require?

C. The compiler generates the following assembly code for st_init:

```
    void st_init(struct test *st)
    st in %rdi
1   st_init:
2     movl    8(%rdi), %eax
3     movl    %eax, 10(%rdi)
4     leaq    10(%rdi), %rax
5     movq    %rax, (%rdi)
6     movq    %rdi, 12(%rdi)
7     ret
```

On the basis of this information, fill in the missing expressions in the code for st_init.

Structure Example

Practice Problem 3.41 (solution page 379)

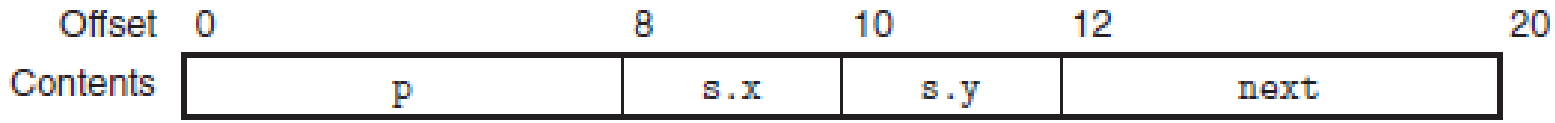
Consider the following structure declaration:

```
struct test {
    short *p;
    struct {
        short x;
        short y;
    } s;
    struct test *next;
};
```

A. What are the offsets (in bytes) of the following fields?

p: _____
s.x: _____
s.y: _____
next: _____

B. How many total bytes does the structure require?



This declaration illustrates that one structure can be embedded within another, just as arrays can be embedded within structures and arrays can be embedded within arrays.

The following procedure (with some expressions omitted) operates on this structure:

```
void st_init(struct test *st) {
    st->s.y  = _________;
    st->p    = _________;
    st->next = _________;
}
```

Structure Example

Practice Problem 3.41 (solution page 379)

Consider the following structure declaration:

```
struct test {
    short *p;
    struct {
        short x;
        short y;
    } s;
    struct test *next;
};
```

This declaration illustrates that one structure can be embedded within another, just as arrays can be embedded within structures and arrays can be embedded within arrays.

The following procedure (with some expressions omitted) operates on this structure:

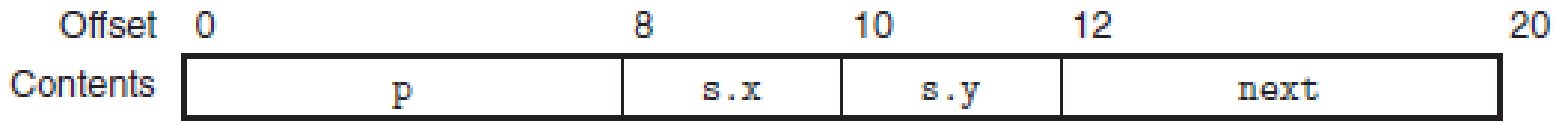
```
void st_init(struct test *st) {
    st->s.y  = _____;
    st->p    = _____;
    st->next = _____;
}
```

A. What are the offsets (in bytes) of the following fields?

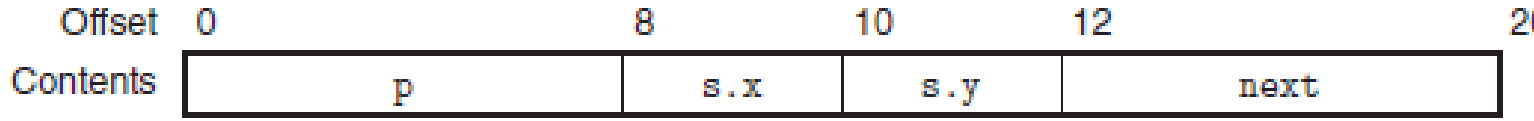
p:	0
s.x:	8
s.y:	10
next:	12

B. How many total bytes does the structure require?

It uses 20 bytes



Structure Example



Practice Problem 3.41 (solution page 379)

Consider the following structure declaration:

```
struct test {
    short *p;
    struct {
        short x;
        short y;
    } s;
    struct test *next;
};
```

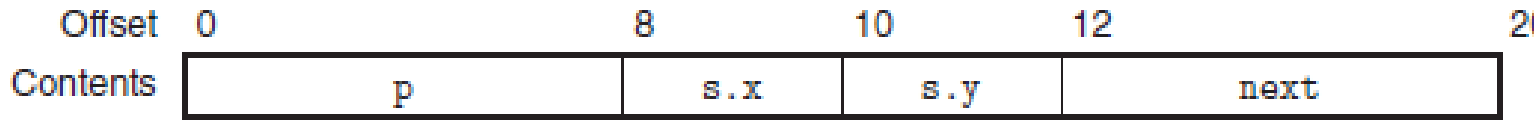
```
void st_init(struct test *st)
st in %rdi
1   st_init:
2     movl    8(%rdi), %eax
3     movl    %eax, 10(%rdi)
4     leaq    10(%rdi), %rax
5     movq    %rax, (%rdi)
6     movq    %rdi, 12(%rdi)
7     ret
```

This declaration illustrates that one structure can be embedded within another, just as arrays can be embedded within structures and arrays can be embedded within arrays.

The following procedure (with some expressions omitted) operates on this structure:

```
void st_init(struct test *st) {
    st->s.y  = _________;
    st->p    = _________;
    st->next = _________;
}
```

Structure Example



Practice Problem 3.41 (solution page 379)

Consider the following structure declaration:

```
struct test {
    short *p;
    struct {
        short x;
        short y;
    } s;
    struct test *next;
};
```

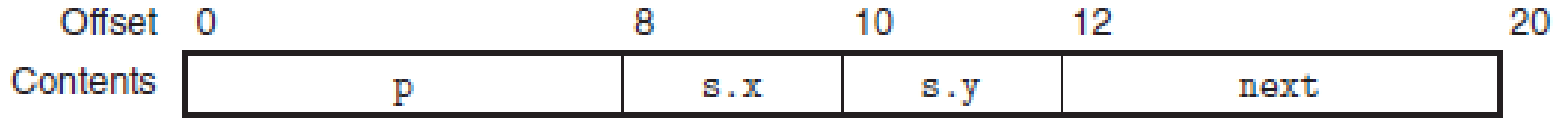
```
void st_init(struct test *st)
st in %rdi
1   st_init:
2     movl    8(%rdi), %eax      Get st->s.x
3     movl    %eax, 10(%rdi)
4     leaq    10(%rdi), %rax
5     movq    %rax, (%rdi)
6     movq    %rdi, 12(%rdi)
7     ret
```

This declaration illustrates that one structure can be embedded within another, just as arrays can be embedded within structures and arrays can be embedded within arrays.

The following procedure (with some expressions omitted) operates on this structure:

```
void st_init(struct test *st) {
    st->s.y  = _________;
    st->p    = _________;
    st->next = _________;
}
```

Structure Example



Practice Problem 3.41 (solution page 379)

Consider the following structure declaration:

```
struct test {
    short *p;
    struct {
        short x;
        short y;
    } s;
    struct test *next;
};
```

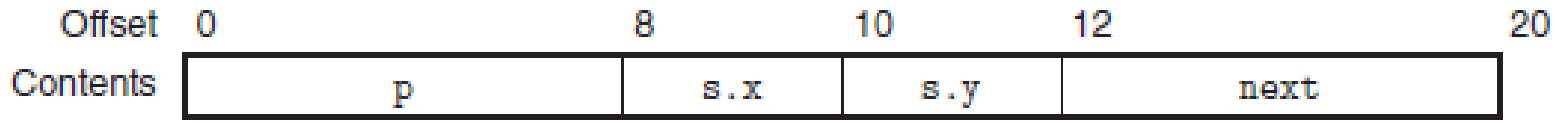
```
void st_init(struct test *st)
st in %rdi
1   st_init:
2     movl    8(%rdi), %eax     Get st->s.x
3     movl    %eax, 10(%rdi)    Save in st->s.y
4     leaq    10(%rdi), %rax
5     movq    %rax, (%rdi)
6     movq    %rdi, 12(%rdi)
7     ret
```

This declaration illustrates that one structure can be embedded within another, just as arrays can be embedded within structures and arrays can be embedded within arrays.

The following procedure (with some expressions omitted) operates on this structure:

```
void st_init(struct test *st) {
    st->s.y  = st->s.x ;
    st->p    = _________;
    st->next = _________;
}
```

Structure Example



Practice Problem 3.41 (solution page 379)

Consider the following structure declaration:

```
struct test {
    short *p;
    struct {
        short x;
        short y;
    } s;
    struct test *next;
};
```

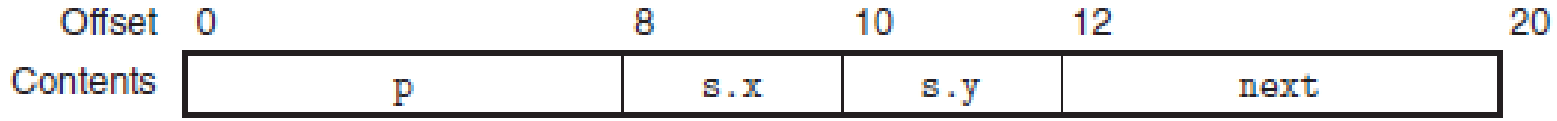
```
void st_init(struct test *st)
st in %rdi
1   st_init:
2     movl    8(%rdi), %eax     Get st->s.x
3     movl    %eax, 10(%rdi)    Save in st->s.y
4     leaq    10(%rdi), %rax    Compute &(st->.sy)
5     movq    %rax, (%rdi)
6     movq    %rdi, 12(%rdi)
7     ret
```

This declaration illustrates that one structure can be embedded within another, just as arrays can be embedded within structures and arrays can be embedded within arrays.

The following procedure (with some expressions omitted) operates on this structure:

```
void st_init(struct test *st) {
    st->s.y  = st->s.x ;
    st->p    = _________;
    st->next = _________;
}
```

Structure Example



Practice Problem 3.41 (solution page 379)

Consider the following structure declaration:

```
struct test {
    short *p;
    struct {
        short x;
        short y;
    } s;
    struct test *next;
};
```

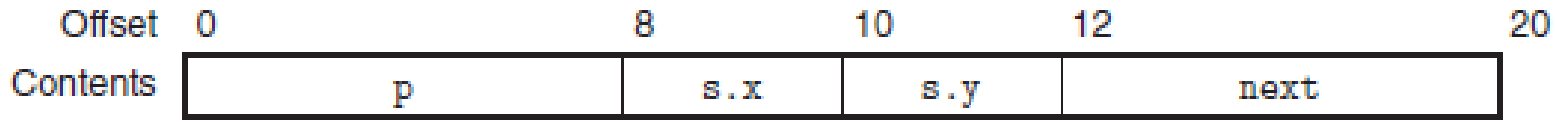
```
void st_init(struct test *st)
st in %rdi
1   st_init:
2     movl    8(%rdi), %eax     Get st->s.x
3     movl    %eax, 10(%rdi)    Save in st->s.y
4     leaq    10(%rdi), %rax    Compute &(st->.sy)
5     movq    %rax, (%rdi)      Store in st->p
6     movq    %rdi, 12(%rdi)
7     ret
```

This declaration illustrates that one structure can be embedded within another, just as arrays can be embedded within structures and arrays can be embedded within arrays.

The following procedure (with some expressions omitted) operates on this structure:

```
void st_init(struct test *st) {
    st->s.y  = st->s.x    ;
    st->p    = &(st->s.y) ;
    st->next = _________  ;
}
```

Structure Example



Practice Problem 3.41 (solution page 379)

Consider the following structure declaration:

```
struct test {
    short *p;
    struct {
        short x;
        short y;
    } s;
    struct test *next;
};
```

```
void st_init(struct test *st)
st in %rdi
1   st_init:
2     movl    8(%rdi), %eax     Get st->s.x
3     movl    %eax, 10(%rdi)    Save in st->s.y
4     leaq    10(%rdi), %rax    Compute &(st->.sy)
5     movq    %rax, (%rdi)      Store in st->p
6     movq    %rdi, 12(%rdi)    Store st in st->next
7     ret
```

This declaration illustrates that one structure can be embedded within another, just as arrays can be embedded within structures and arrays can be embedded within arrays.

The following procedure (with some expressions omitted) operates on this structure:

```
void st_init(struct test *st) {
    st->s.y  = st->s.x    ;
    st->p    = &(st->s.y) ;
    st->next = st;        ;
}
```

Structure Example

Practice Problem 3.42 (solution page 379)

The following code shows the declaration of a structure of type ACE and the prototype for a function test:

```
struct ACE {
    short      v;
    struct ACE *p;
};

short test(struct ACE *ptr);
```

When the code for fun is compiled, GCC generates the following assembly code:

```
    short test(struct ACE *ptr)
    ptr in %rdi
1   test:
2     movl    $1, %eax
3     jmp     .L2
4   .L3:
5     imulq   (%rdi), %rax
6     movq    2(%rdi), %rdi
7   .L2:
8     testq   %rdi, %rdi
9     jne     .L3
10    rep; ret
```

A. Use your reverse engineering skills to write C code for test.

B. Describe the data structure that this structure implements and the operation performed by test

Structure Example

```
    short test(struct ACE *ptr)
    ptr in %rdi
1   test:
2     movl    $1, %eax
3     jmp     .L2
4   .L3:
5     imulq   (%rdi), %rax
6     movq    2(%rdi), %rdi
7   .L2:
8     testq   %rdi, %rdi
9     jne     .L3
10    rep; ret
```

In ACE;
v occupies 2 bytes
*p occupies 8 bytes

Ptr in %rdi
Result in %rax

Structure Example

```
    short test(struct ACE *ptr)
    ptr in %rdi
1   test:
2     movl    $1, %eax
3     jmp     .L2
4   .L3:
5     imulq   (%rdi), %rax
6     movq    2(%rdi), %rdi
7   .L2:
8     testq   %rdi, %rdi
9     jne     .L3
10    rep; ret
```

result = 1
Goto middle

Middle:

In ACE;
v occupies 2 bytes
*p occupies 8 bytes

Ptr in %rdi
Result in %rax

Structure Example

```
    short test(struct ACE *ptr)
    ptr in %rdi
1   test:
2     movl    $1, %eax
3     jmp     .L2
4   .L3:
5     imulq   (%rdi), %rax
6     movq    2(%rdi), %rdi
7   .L2:
8     testq   %rdi, %rdi
9     jne     .L3
10    rep; ret
```

result = 1
Goto middle
Loop :
  result= result * ptr->v
  ptr = ptr->p
Middle:

In ACE;
v occupies 2 bytes
*p occupies 8 bytes

Ptr in %rdi
Result in %rax

Structure Example

```
    short test(struct ACE *ptr)
    ptr in %rdi
1   test:
2     movl    $1, %eax
3     jmp     .L2
4   .L3:
5     imulq   (%rdi), %rax
6     movq    2(%rdi), %rdi
7   .L2:
8     testq   %rdi, %rdi
9     jne     .L3
10    rep; ret
```

result = 1
Goto middle
Loop :
  result= result * ptr->v
  ptr = ptr->p
Middle:
Test ptr
If ptr is not null, go to loop

In ACE;
v occupies 2 bytes
*p occupies 8 bytes

Ptr in %rdi
Result in %rax

Structure Example

```
    short test(struct ACE *ptr)
    ptr in %rdi
1   test:
2     movl    $1, %eax
3     jmp     .L2
4   .L3:
5     imulq   (%rdi), %rax
6     movq    2(%rdi), %rdi
7   .L2:
8     testq   %rdi, %rdi
9     jne     .L3
10    rep; ret
```

result = 1
Goto middle
Loop :
  result= result * ptr->v
  ptr = ptr->p
Middle:
Test ptr
If ptr is not null, go to loop

In ACE;
v occupies 2 bytes
*p occupies 8 bytes

Ptr in %rdi
Result in %rax

```
short test(struct ACE *ptr) {
    short val = 1;
    while (ptr) {
        val *= ptr->v;
        ptr  = ptr->p;
    }
    return val;
}
```

Structure Example

```
    short test(struct ACE *ptr)
    ptr in %rdi
1   test:
2     movl    $1, %eax
3     jmp     .L2
4   .L3:
5     imulq   (%rdi), %rax
6     movq    2(%rdi), %rdi
7   .L2:
8     testq   %rdi, %rdi
9     jne     .L3
10    rep; ret
```

result = 1
Goto middle
Loop :
  result= result * ptr->v
  ptr = ptr->p
Middle:
Test ptr
If ptr is not null, go to loop

In ACE;
v occupies 2 bytes
*p occupies 8 bytes

Ptr in %rdi
Result in %rax

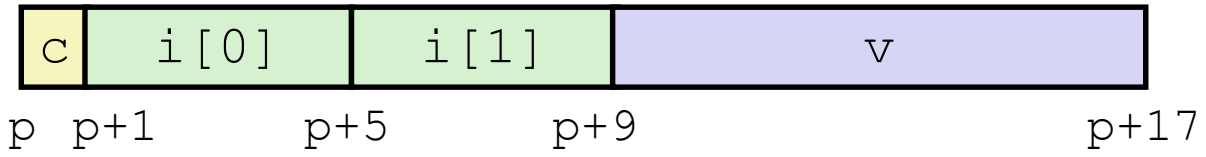
```
short test(struct ACE *ptr) {
    short val = 1;
    while (ptr) {
        val *= ptr->v;
        ptr  = ptr->p;
    }
    return val;
}
```

A. Describe the data structure that this structure implements and the operation performed by test.

Each structure is an element in a singly linked list, with field v that contains the value of element and p which points to the next element.
Function fun computes the multiplication of the element values in the list.

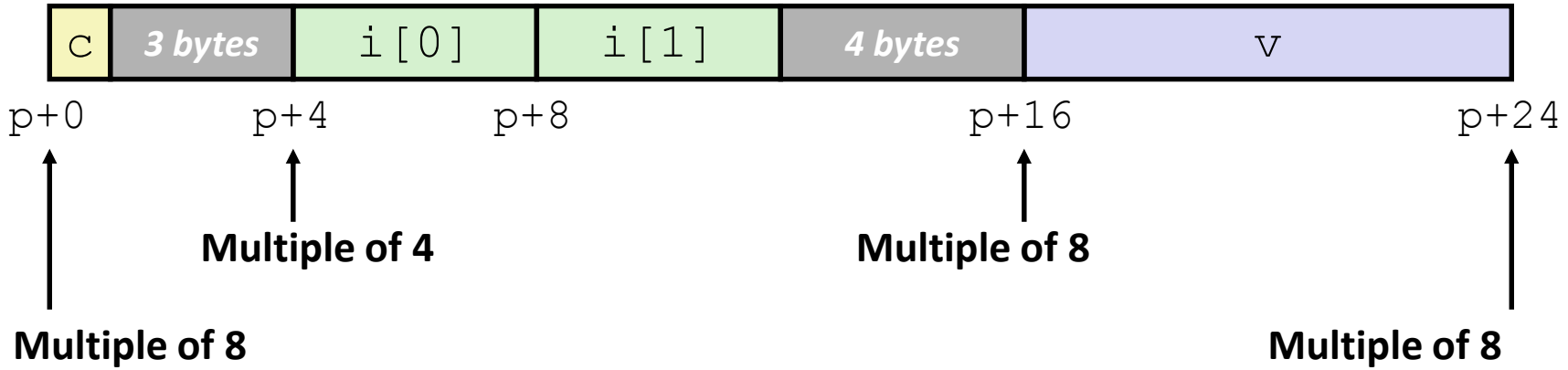
Structures & Alignment

- **Unaligned Data**




```
struct S1 {
  char c;
  int i[2];
  double v;
} *p;
```

- **Aligned Data**
  - Primitive data type requires ***K*** bytes
  - Address must be multiple of ***K***



Alignment Principles

- **Aligned Data**
  - Primitive data type requires ***K*** bytes
  - Address must be multiple of ***K***
  - Required on some machines; advised on x86-64
- **Motivation for Aligning Data**
  - Memory accessed by (aligned) chunks of 4 or 8 bytes (system dependent)
    - Inefficient to load or store datum that spans quad word boundaries
    - Virtual memory trickier when datum spans 2 pages
- **Compiler**
  - Inserts gaps in structure to ensure correct alignment of fields

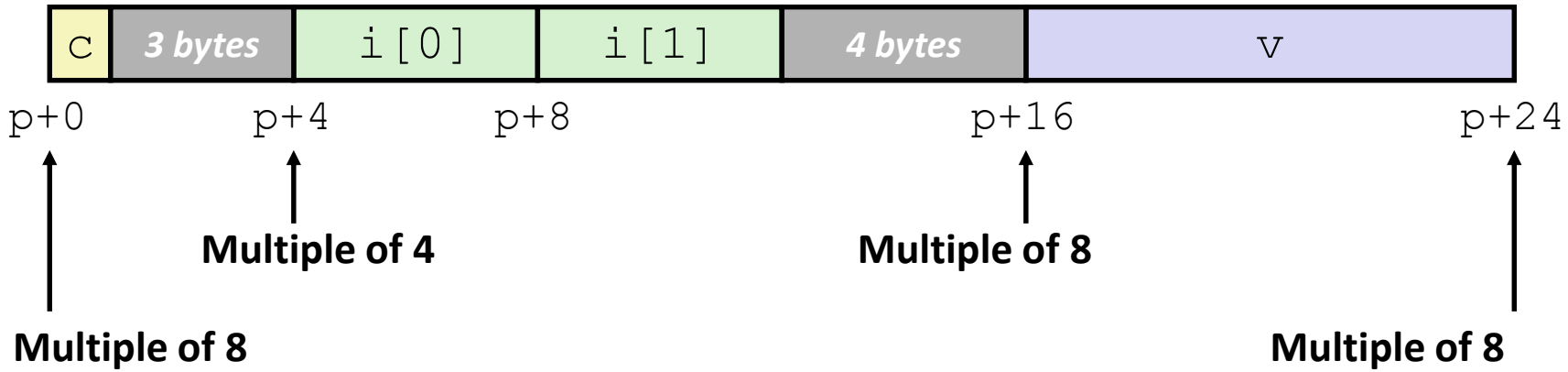
Specific Cases of Alignment (x86-64)

- **1 byte: `char`, …**
  - no restrictions on address
- **2 bytes: `short`, …**
  - lowest 1 bit of address must be 0_2
- **4 bytes: `int`, `float`, …**
  - lowest 2 bits of address must be 00_2
- **8 bytes: `double`, `long`, `char *`, …**
  - lowest 3 bits of address must be 000_2

Satisfying Alignment with Structures

- **Within structure:**
  - Must satisfy each element's alignment requirement
- **Overall structure placement**
  - Each structure has alignment requirement **K**
    - **K** = Largest alignment of any element
  - Initial address & structure length must be multiples of **K**
- **Example:**
  - K = 8, due to `double` element

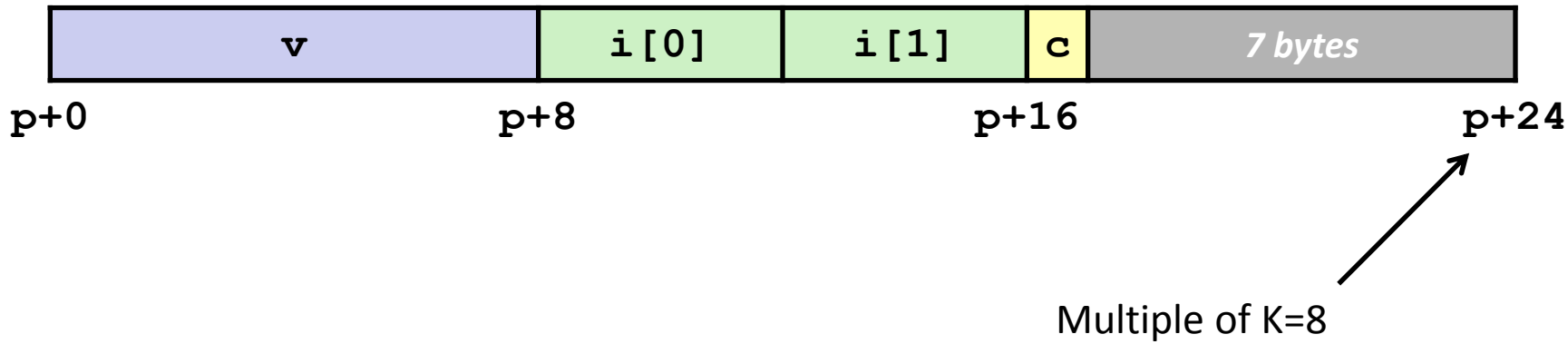
```
struct S1 {
  char c;
  int i[2];
  double v;
} *p;
```



Meeting Overall Alignment Requirement

- For largest alignment requirement K
- Overall structure must be multiple of K

```
struct S2 {
  double v;
  int i[2];
  char c;
} *p;
```



Structure Alignment Example

Practice Problem 3.44 (solution page 381)

For each of the following structure declarations, determine the offset of each field, the total size of the structure, and its alignment requirement for x86-64:

A. `struct P1 { short i; int c; int *j; short *d; };`

B. `struct P2 { int i[2]; char c[8]; short s[4]; long *j; };`

C. `struct P3 { long w[2]; int *c[2] };`

D. `struct P4 { char w[16]; char *c[2] };`

E. `struct P5 { struct P4 a[2]; struct P1 t };`

Structure Alignment Example

Practice Problem 3.44 (solution page 381)

For each of the following structure declarations, determine the offset of each field, the total size of the structure, and its alignment requirement for x86-64:

A. `struct P1 { short i; int c; int *j; short *d; };`

B. `struct P2 { int i[2]; char c[8]; short s[4]; long *j; };`

C. `struct P3 { long w[2]; int *c[2] };`

D. `struct P4 { char w[16]; char *c[2] };`

E. `struct P5 { struct P4 a[2]; struct P1 t };`

A. `struct P1 { short i; int c; int *j; short *d; };`

i	c	j	d	Total	Alignment
0	2	6	14	22	8

Structure Alignment Example

Practice Problem 3.44 (solution page 381)

For each of the following structure declarations, determine the offset of each field, the total size of the structure, and its alignment requirement for x86-64:

A. `struct P1 { short i; int c; int *j; short *d; };`

B. `struct P2 { int i[2]; char c[8]; short s[4]; long *j; };`

C. `struct P3 { long w[2]; int *c[2] };`

D. `struct P4 { char w[16]; char *c[2] };`

E. `struct P5 { struct P4 a[2]; struct P1 t };`

B. `struct P2 { int i[2]; char c[8]; short [4]; long *j; };`

i	c	d	j	Total	Alignment
0	8	16	24	32	8

Structure Alignment Example

Practice Problem 3.44 (solution page 381)

For each of the following structure declarations, determine the offset of each field, the total size of the structure, and its alignment requirement for x86-64:

A. `struct P1 { short i; int c; int *j; short *d; };`

B. `struct P2 { int i[2]; char c[8]; short s[4]; long *j; };`

C. `struct P3 { long w[2]; int *c[2] };`

D. `struct P4 { char w[16]; char *c[2] };`

E. `struct P5 { struct P4 a[2]; struct P1 t };`

C. `struct P3 { long w[2]; int *c[2] };`

w	c	Total	Alignment
0	16	32	8

Structure Alignment Example

Practice Problem 3.44 (solution page 381)

For each of the following structure declarations, determine the offset of each field, the total size of the structure, and its alignment requirement for x86-64:

A. `struct P1 { short i; int c; int *j; short *d; };`

B. `struct P2 { int i[2]; char c[8]; short s[4]; long *j; };`

C. `struct P3 { long w[2]; int *c[2] };`

D. `struct P4 { char w[16]; char *c[2] };`

E. `struct P5 { struct P4 a[2]; struct P1 t };`

D. `struct P4 { char w[16]; char *c[2] };`

w	c	Total	Alignment
0	16	32	8

Structure Alignment Example

Practice Problem 3.44 (solution page 381)

For each of the following structure declarations, determine the offset of each field, the total size of the structure, and its alignment requirement for x86-64:

A. `struct P1 { short i; int c; int *j; short *d; };`

B. `struct P2 { int i[2]; char c[8]; short s[4]; long *j; };`

C. `struct P3 { long w[2]; int *c[2] };`

D. `struct P4 { char w[16]; char *c[2] };`

E. `struct P5 { struct P4 a[2]; struct P1 t };`

E. `struct P5 { struct P4 a[2]; struct P1 t };`

a	t	Total	Alignment
0	64	86	8

Structure Alignment Example

Practice Problem 3.45 (solution page 381)

Answer the following for the structure declaration

```
struct {
  int    *a;
  float  b;
  char   c;
  int    d;
  long   e;
  double f;

  int    g;
  char   *h;
} rec;
```

A. What are the byte offsets of all the fields in the structure?

B. What is the total size of the structure?

C. Rearrange the fields of the structure to minimize wasted space, and then show the byte offsets and total size for the rearranged structure.

Structure Alignment Example

Practice Problem 3.45 (solution page 381)

Answer the following for the structure declaration

```
struct {
  int    *a;
  float  b;
  char   c;
  int    d;
  long   e;
  double f;
  int    g;
  char   *h;
} rec;
```

A. Here are the object sizes and byte offsets:

Field	a	b	c	d	e	f	g	h
Size	8	4	1	4	8	8	4	8
Offset	0	8	12	16	24	32	40	48

Structure Alignment Example

Practice Problem 3.45 (solution page 381)

Answer the following for the structure declaration

```
struct {
  int    *a;
  float  b;
  char   c;
  int    d;
  long   e;
  double f;
  int    g;
  char   *h;
} rec;
```

A. Here are the object sizes and byte offsets:

Field	a	b	c	d	e	f	g	h
Size	8	4	1	4	8	8	4	8
Offset	0	8	12	16	24	32	40	48

B. The structure is a total of 56 bytes long. The end of the structure does not require padding to satisfy the 8-byte alignment requirement.

Structure Alignment Example

Practice Problem 3.45 (solution page 381)

Answer the following for the structure declaration

```
struct {
  int    *a;
  float  b;
  char   c;
  int    d;
  long   e;
  double f;
  int    g;
  char   *h;
} rec;
```

```
struct {
    int    *a;
    char   *h;
    double f;
    long   e;
    float  b;
    int    g;
    int    d;
    char   c;
} rec;
```

C. One strategy that works, when all data elements have a length equal to a power of 2, is to order the structure elements in descending order of size. This leads to a declaration:

	Field							
	a	h	f	e	b	g	d	c
Size	8	8	8	8	4	4	4	1
Offset	0	8	16	24	32	36	40	44

The structure must be padded by 3 bytes to satisfy the 8-byte alignment requirement, giving a total of 48 bytes.